=

Course	COMP 8005
Program	Bachelor of Science in Applied Computer Science
Term	January 2026

- This is an individual programming assignment.

Objective

Design and implement a distributed password cracking system for UNIX systems using a controller / multi-worker architecture.

You will:

- Measure performance and overheads of distribution.
- Study scaling across multiple workers (inter-host) and threads within each worker (intra-worker).
- Use measured results for 1–3 workers to predict performance at 5 workers, then validate by running 5 workers.

System Requirements

The system must consist of:

- A controller program running on Host A.
- N worker programs running on other hosts.

Worker Threads

Each worker is multithreaded and supports a configurable thread count via the command line.

Fault Model

You do not need to handle worker crashes.

- If a worker crashes, their assigned work is lost.
- The controller must not reassign lost chunks.

Learning Outcomes

By completing this assignment, you will be able to:

- Design and implement a controller / multi-worker distributed system
- Implement worker registration, remote job execution, and work assignment

- Correctly parse UNIX shadow entries and verify candidate passwords
- Implement a multi-threaded search with correct partitioning and no duplicate coverage
- Implement controller-driven heartbeat and progress reporting
- Measure and analyze end-to-end distributed execution time and overhead
- Measure and analyze scaling behaviour for 1–3 workers and validate a 5-worker prediction
- Work with multiple UNIX password hashing algorithms
- Present experimental results using tables, graphs, and written analysis

Supported Password Hashing Algorithms

Your system must support all of the following UNIX password hashing algorithms:

- yescrypt
- bcrypt
- SHA256
- SHA512
- MD5

Password Search Parameters

All experiments must use:

- A 3-character password
- The 79-character legal character set

The entire search space must be covered exactly once across the distributed system.

Updated System Architecture

Controller (Host A)

There is exactly one controller. The controller is responsible for:

1. Accepting a shadow file and username from the command line
2. Parsing the shadow file entry to extract:
    - hashing algorithm identifier
    - salt
    - password hash
3. Accepting registrations from multiple workers
4. Sending the full cracking job parameters to workers (algorithm, salt, hash, starting password, ending password - starting/ending can be passwords, IDs or some other thing that makes sense for your implementation). (the chunk size is configurable via the command line)

5. Managing a global chunk allocator that assigns disjoint chunks of the search space to workers
6. Sending a heartbeat request every N seconds (configurable via the command line)
7. Receiving from workers:
    - periodic heartbeat responses (progress)
    - completion notices for chunks (optional but recommended)
    - final result (password found) or exhaustion status
8. Enforcing global stop:
    - once any worker finds the password, no new chunks are issued
    - controller broadcasts/returns STOP to workers so they can exit
9. Reporting:
    - final result
    - timing breakdowns
    - experimental measurements

Workers (Host(s) B…)

There are N workers, each responsible for:

1. Starting a service and registering with the controller
2. Receiving job parameters
3. Repeatedly requesting work (pull model):
    - ask for a chunk
    - crack the chunk using T threads
    - report FOUND (immediately) or request the next chunk
4. Responding to the controller's heartbeats with progress since the last heartbeat
5. Exiting when: the password is found by any worker (STOP message)

Work Assignment Model

Distribution level (between hosts)

The controller partitions the global search space into chunks.

- Each chunk corresponds to N candidate passwords (configurable).
- A chunk is issued at most once.
- Workers request a new chunk when done.

Correctness Requirements (Global)

Your system must guarantee:

- No password can be tried twice (no duplicate candidate attempts globally)
- No omissions unless caused by a worker crash (which is allowed under this fault model)

Worker level (within a worker process)

Within a worker, the assigned chunk must be partitioned across T threads such that:

- Each candidate in the chunk is tested exactly once
- No duplicates
- No omissions

Acceptable thread partitioning models include:

- Static partitioning: each thread gets a disjoint subrange of the chunk
- Dynamic partitioning (preferred): a thread-safe work index / internal chunking scheme

Heartbeat Protocol

The heartbeat provides liveness + progress visibility during long runs.

Behavior:

- The controller sends a heartbeat request every N seconds.
- Each worker replies with the number of candidates tested since the last heartbeat.

Minimum required heartbeat response payload:

- delta_tested: number of candidate attempts since the last heartbeat

Recommended (optional but useful) fields:

- total_tested
- threads_active
- current_rate (attempts/sec since last heartbeat)
- current_chunk_id or (chunk_start, chunk_count)

Important: The heartbeat must not distort correctness. It is for measurement/observability, not control.

Stop Conditions

- If any worker finds the password:
    - The worker reports FOUND to the controller
    - The controller sets a found flag
    - No new chunks are assigned
    - Workers receive STOP and exit

Command Line Interface

Controller

The controller must be runnable as:

```
controller -f <shadow file> -u <username> -p <port> -b
<heartbeat_seconds> -c <chunk_size>
```

Required arguments:

- -f path to shadow file (/etc/shadow or a copy)
- -u username whose password will be cracked
- -p port number to listen on
- -b heartbeat interval in seconds
- -c chunk size in candidates (passwords) per assigned chunk

Worker

Each worker must be runnable as:

```
worker -c <controller_host> -p <port> -t <threads>
```

Required arguments:

- -c controller hostname or IP address
- -p controller port number
- -t number of worker threads to use for cracking

Performance Experiments

Required worker counts

You must run the full experiment suite at:

- 1 worker
- 2 workers
- 3 workers
- 5 workers (prediction validation)

Required thread count per worker
Use a fixed thread count per worker for worker-scaling experiments:

- t = 1 (pure distribution scaling) or
- t = the number of cores in the CPU - 1

Required measurements (per trial)

For each hashing algorithm and worker count, measure and report:

- controller-side parsing time
- job dispatch/registration overhead
- chunk assignment overhead (per chunk and/or total)
- worker cracking time (compute/search)
- result return latency (worker → controller)
- total end-to-end runtime
- heartbeat observations (progress per heartbeat; compute rate stability/variance)

Repetition and summary statistics

Run each experiment multiple times and report summary statistics:

- mean
- min/max or standard deviation

Prediction Requirement (Worker Scaling)

After measuring 1–3 workers:

1. Predict what will happen at 5 workers (runtime and/or speedup).
2. Explain your model.

Acceptable approaches include:

- Amdahl-style model using measured serial fraction (controller overhead + networking)
- Throughput fits with diminishing returns
- Overhead-aware model including chunk assignment, coordination, and scheduling costs

Then:

- Run the experiment with 5 workers
- Graph the measured result and explain why it did or did not match your prediction
- If it diverges, identify the causes (controller bottleneck, network contention, chunk size too small/large, synchronization overhead within workers, OS scheduling, etc.).

Constraints

- Exactly one controller
- Multiple workers; workers request work in chunks
- No shared memory between hosts
- Workers use T threads (configurable)
- No password is attempted twice globally
- Minimize unnecessary communication
- Heartbeat is required and must not break correctness

- Correctness across all supported hashing algorithms

Resources

- UNIX crypt(3) documentation
- Password hashing libraries for your chosen language
- Socket/network programming documentation
- Profiling tools (time, perf, htop)

Submission

- Ensure your submission meets all the guidelines, including formatting, file type, and submission.
- Follow the AI usage guidelines.
- Be aware of the late submission policy to avoid losing marks.
- ***Note: Please strictly adhere to the submission requirements to ensure you don't lose any marks.***

Evaluation

Topic	**Value**
Design	10
Testing	25
Implementation	55
Report	10
Total	100

Testing Information

You should test with a password and algorithm - but be aware that when you do your final demo of the password cracker, you will not know:

- Password length
- Password hashing algorithm
- Password